HideText: A Deterministic Next-Token Text Steganography Testbed for Local Language Models

Anonymous Author(s)

ABSTRACT

We present HideText, a reproducible engineering testbed for text steganography based on next-token distributions from local language models. In HideText, the sender encrypts a secret, converts the resulting packet into a finite interval, and selects each generated token by intersecting that interval with a quantized candidate distribution derived from the model. The receiver replays the same model, tokenizer, prompt, seed, and codec configuration and mirrors the interval updates to recover the secret. Our primary design goal is not maximum capacity or anti-detection strength; instead, it is deterministic end-to-end decodability under tightly controlled conditions, including bilingual prompts and outputs. The current implementation combines authenticated encryption, explicit packet framing, deterministic candidate selection, integer quantization, and an integer finite-interval header/body codec. It includes a fast bilingual toy backend for protocol regression tests and a local `llama.cpp` backend for CPU-based experiments with a quantized Qwen model. We also introduce a low-entropy retry mechanism that abandons an encoding attempt when the rolling average candidate entropy collapses below a threshold, then retries with a fresh randomized packet. The resulting system is a practical platform for studying deterministic text steganography under local-model constraints while making explicit the assumptions and failure modes that such systems require.

CCS CONCEPTS

• **Security and privacy** → **Systems security**; *Cryptography*; • **Computing methodologies** → *Natural language generation.*

KEYWORDS

text steganography, language models, arithmetic coding, deterministic decoding, reproducibility

ACM Reference Format:
Anonymous Author(s). 2026. HideText: A Deterministic Next-Token Text Steganography Testbed for Local Language Models. In . ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Natural-language steganography attempts to embed hidden information in fluent text while keeping the resulting text apparently innocuous. Classical text steganography often modifies a finished cover text or relies on ad hoc textual artifacts, but modern neural approaches can instead treat language generation itself as the covert channel. In these systems, the sender uses a language model's token distribution to decide which next token to emit, and the receiver replays the same distribution sequence to decode the embedded message. Prior work has explored neural linguistic steganography with recurrent models and neural generators [2, 5–7].

HideText focuses on a specific engineering problem inside this space: how to build a deterministic, end-to-end reproducible text steganography demo for local open models. We target a setting in which both endpoints can share the exact model weights, tokenizer, prompt template, seed, and codec implementation. Under that assumption, the project favors fail-closed behavior and protocol determinism over stealth optimality, robustness to editing, or capacity maximization.

This emphasis is motivated by practice. Once a steganographic language pipeline mixes floating-point probability manipulation, candidate truncation, encryption, prompt templating, and model inference, a large number of seemingly small implementation choices become protocol-critical. If sender and receiver disagree about any of them, decoding can drift silently. HideText therefore treats determinism as a first-class systems property: candidate selection is deterministic, quantization is integer based, packet framing is explicit, cryptographic metadata is authenticated, and mismatched runtime configurations are rejected rather than tolerated.

The paper makes three contributions:

(1) We describe a deterministic architecture for text steganography that combines authenticated encryption, explicit packet framing, deterministic candidate selection, integer probability quantization, and an integer finite-interval header/body codec.
(2) We present a bilingual local-model implementation with both a deterministic toy backend and a CPU-based `llama.cpp` backend for a quantized Qwen model.
(3) We introduce a practical low-entropy retry mechanism for cases where the model collapses into a near-deterministic region and temporarily ceases to provide enough entropy to complete the embedding process.

2 THREAT MODEL AND GOALS

HideText assumes a passive observer who can read the generated text but does not know the sender and receiver's shared key or full configuration, and does not actively modify the text in transit. The system is designed for deterministic recovery under shared conditions rather than for robustness against editing, paraphrasing, or active steganalysis.

The current goals are:

- deterministic sender–receiver agreement;
- end-to-end plaintext recovery under matched configurations;

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

- bilingual operation for Chinese and English prompts;
- practical local execution with small open models.

The current non-goals are:

- resistance to a dedicated steganalyst;
- recovery after token insertion, deletion, or paraphrasing;
- compatibility with black-box APIs that expose only partial log-probabilities;
- theoretical capacity optimality.

3 SYSTEM OVERVIEW

Figure-free by design, HideText can be understood as a six-layer pipeline.

3.1 Crypto and Packet Framing

The user plaintext is first encoded as UTF-8 and then encrypted with `scrypt` for key derivation [3] and `ChaCha20-Poly1305` for authenticated encryption [1]. The ciphertext is packaged as a fixed-format stego packet with a magic value, protocol version, cryptographic sizing metadata, ciphertext length, and a 64-bit runtime fingerprint.

The packet fingerprint binds the runtime protocol dictionary, backend metadata, and a hash of the shared prompt. This means that prompt drift, seed drift, tokenizer drift, or backend drift triggers a fail-closed error rather than "best effort" decoding.

3.2 Model Backend

HideText currently ships with two backends. The first is a deterministic bilingual toy character backend for fast regression tests. The second is a local `llama.cpp`-based backend that extracts next-token logits from a quantized Qwen model on CPU. The real backend fixes the prompt template and blocks special tokens that would compromise replay determinism.

3.3 Candidate Policy and Quantization

At each generation step, HideText sorts candidates by descending probability with token-id tie breaking, applies top-p truncation together with a maximum candidate cap, and renormalizes the remaining mass. It then quantizes the resulting probabilities into integer frequencies with total frequency T, ensuring that all retained candidates receive at least one frequency count and that sender and receiver obtain the same cumulative distribution function.

This separation is deliberate. Arithmetic-style codecs operate most safely over integer cumulative distributions, not over unconstrained floating-point probabilities [4].

3.4 Finite-Interval Header/Body Codec

The core codec treats a segment of n bytes as an integer point inside $[0, 2^{8n})$. At each step, the current interval is partitioned according to the quantized cumulative distribution, and the sender selects the token whose sub-interval contains the target point. The receiver mirrors the same interval updates after observing the emitted token sequence. Once the interval width reaches one, the segment is uniquely determined.

HideText uses two segments:

- a fixed-size header segment;
- a body segment whose exact length is learned after decoding the header.

This design avoids the need to encode an unknown-length packet monolithically.

4 DETERMINISM AS A PROTOCOL PROPERTY

Many components that might otherwise look like implementation details are protocol-defining in HideText:

- the prompt and its template;
- backend metadata, including model identity and tokenizer hash;
- seed and generation configuration;
- candidate truncation, ordering, and token masking;
- probability renormalization and integer quantization;
- interval update rules and termination conditions.

This perspective is consistent with prior observations that replay-based linguistic steganography is brittle when segmentation or tokenization differs between endpoints [2]. HideText responds by rejecting mismatches early and by structuring the implementation so that protocol-sensitive logic is pure, serializable, and easy to test.

5 LOW-ENTROPY RETRY MECHANISM

In practice, local language models can enter low-entropy regions in which the next-token distribution becomes almost deterministic. In the extreme case, the candidate set collapses to a single token and the instantaneous embedding capacity becomes zero. Existing stall detectors can catch prolonged lack of progress, but they may do so late: a model can remain technically "alive" while still offering negligible embedding capacity.

HideText therefore adds a rolling low-entropy detector. During encoding, the sender tracks the mean candidate entropy over a fixed-size window of consecutive generation steps. With the current defaults, if the rolling average across 32 steps falls below 0.1 bit, the sender abandons the current attempt. It then rebuilds the packet with a fresh random salt and nonce and restarts encoding. Because HideText uses encrypt-then-stego, the packet body changes across attempts even when the plaintext, prompt, and seed stay fixed.

This mechanism is not part of the decode fingerprint. It is operational sender-side logic for escaping bad paths rather than shared protocol state. After three failed attempts, HideText raises an actionable error recommending a different cover prompt or a shorter secret.

The retry logic has an important limitation: for equal message lengths and fixed configuration, the packet header remains stable across retries, while the packet body changes. Consequently, retries are most useful when the low-entropy collapse happens during or after body encoding. A pathological collapse during header encoding still fails closed.

6 IMPLEMENTATION

The reference implementation is organized into distinct modules for configuration, packet framing, cryptography, model backends, candidate selection, quantization, codec logic, and end-to-end encode/decode pipelines. This separation makes it easier to reason

Table 1: Current verification status of the implementation.

Scope	Evidence
Toy backend	The repository test suite currently contains 23 automated tests covering codec behavior, packet framing, cryptography, deterministic quantization, bilingual round trips, CLI behavior, prompt drift, seed drift, text mutation, stall detection, and low-entropy retry exhaustion.
Real backend	A CPU-based smoke test passes with a downloaded GGUF build of `Qwen/Qwen3-4B-Instruct-2507` through the `llama.cpp` backend, confirming a real-model encode/decode round trip under fixed prompt, seed, salt, and nonce.

about which changes alter the protocol and which changes merely alter local operational policy.

The system currently supports:

- bilingual toy prompts and outputs for protocol testing;
- a CPU path for `Qwen/Qwen3-4B-Instruct-2507` via GGUF and `llama.cpp`;
- CLI entry points for encoding, decoding, evaluation, progress reporting, and file-based inputs;
- automated tests for packet logic, cryptography, quantization, codec correctness, bilingual round trips, failure cases, and low-entropy retry behavior.

7 CURRENT EVALUATION STATUS

HideText is presently an engineering demo, not a full empirical study. Accordingly, we report verification evidence rather than broad performance claims.

These checks establish that the implementation can complete end-to-end recovery in both a fast deterministic setting and a realistic local-model setting. They do not yet establish detection resistance, optimal capacity, or robustness under edited text.

Future evaluation should measure:

- bits per token and bits per character across message lengths;
- the fraction of source entropy actually converted into payload capacity;
- runtime and tokens-per-second for encoding and decoding on CPU;
- the frequency and cost of low-entropy retries under different prompts and candidate policies;
- changes in text quality under both toy and real backends;
- sensitivity to prompt design in both Chinese and English.

8 LIMITATIONS

HideText intentionally omits several properties that a deployment-grade system would require.

- It assumes a passive observer and does not handle text editing, truncation, or active channel tampering.
- It requires exact agreement on model version, tokenizer, prompt template, seed, and protocol configuration.
- It currently prioritizes deterministic replay over high capacity and over naturalness optimization.
- It does not yet incorporate forward error correction or robustness coding.
- The real-model validation is currently a smoke test rather than a large benchmark campaign.

These limitations are deliberate. The project is meant to be a stable testbed for deterministic next-token steganography experiments, not a claim of production-ready secure messaging.

9 RELATED WORK

Neural text steganography has progressed from recurrent generators that directly encode bitstreams into generated text [5] to language-model-based schemes that interpret next-token distributions as a channel [7]. Subsequent work has studied stronger formalizations and security notions for generative linguistic steganography [6], as well as practical decoding issues caused by segmentation ambiguity in modern tokenization schemes [2].

HideText is complementary to this line of work. Rather than proposing a new theoretical security proof or a novel steganographic objective, it isolates the systems-level requirements for deterministic replay with local open models. In that sense, it is closer to an executable protocol workbench than to a new stegosystem proposal. The codec core also inherits the engineering motivation behind arithmetic coding: explicitly separating the probabilistic model from the integer channel encoder [4].

10 CONCLUSION

HideText demonstrates that next-token text steganography can be packaged as a deterministic, bilingual, locally executable engineering stack. By combining encrypt-then-stego packet construction, deterministic candidate selection, integer quantization, an integer finite-interval codec, and strict fail-closed checks, the system provides a reproducible baseline for future experiments. The added low-entropy retry mechanism addresses a practical failure mode observed in local models without changing the shared decode protocol. We hope this testbed makes it easier to study the boundary between reproducible text generation and covert communication under realistic local-model constraints.

REFERENCES

[1] Yoav Nir and Adam Langley. 2018. ChaCha20 and Poly1305 for IETF Protocols. RFC 8439. https://doi.org/10.17487/RFC8439

[2] Jumon Nozaki and Yugo Murawaki. 2022. Addressing Segmentation Ambiguity in Neural Linguistic Steganography. https://doi.org/10.48550/arXiv.2211.06662 arXiv:2211.06662

[3] Colin Percival and Simon Josefsson. 2016. The scrypt Password-Based Key Derivation Function. RFC 7914. https://doi.org/10.17487/RFC7914

[4] Ian H. Witten, Radford M. Neal, and John G. Cleary. 1987. Arithmetic Coding for Data Compression. *Commun. ACM* 30, 6 (1987), 520–540. https://doi.org/10.1145/214762.214771

[5] Zhongliang Yang, Xiaoqing Guo, Ziming Chen, Yongfeng Huang, and Yujin Zhang. 2018. RNN-Stega: Linguistic Steganography Based on Recurrent Neural Networks. *IEEE Transactions on Information Forensics and Security* 14, 5 (2018), 1280–1295. https://doi.org/10.1109/TIFS.2018.2871746

[6] Siyu Zhang, Zhongliang Yang, Jinshuai Yang, and Yongfeng Huang. 2021. Provably Secure Generative Linguistic Steganography. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. 3046–3055. https://doi.org/10.18653/v1/2021.findings-acl.268

[7] Zachary M. Ziegler, Yuntian Deng, and Alexander M. Rush. 2019. Neural Linguistic Steganography. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural*

Language Processing (EMNLP-IJCNLP). 1210–1215. https://doi.org/10.18653/v1/D19-1115

A OPEN SCIENCE

The HideText artifact consists of source code, automated tests, protocol documentation, and scripts for running toy and local-model evaluations. For an anonymous submission, the artifact should be provided through an anonymous archival URL that contains:

- the full source tree;
- the exact dependency installation commands;
- the test commands for the toy backend and the real-model smoke test;
- instructions for downloading the GGUF model used in the local experiment;
- the prompts and configuration files needed to reproduce the reported smoke tests.

The current real-model experiment depends on an externally hosted model file, so the artifact should include the download recipe and the exact filename rather than redistributing the weights directly unless licensing permits redistribution.

B ETHICAL CONSIDERATIONS

Text steganography is dual use. On the positive side, deterministic covert channels can support censorship circumvention research, privacy-preserving communication, and the study of machine-generated cover channels. On the negative side, the same techniques could be adapted for covert exfiltration or concealment of malicious coordination.

HideText attempts to reduce misuse risk in three ways. First, it is explicitly positioned as a local research demo rather than a production messaging system. Second, it prioritizes reproducibility and controlled experimentation over stealth claims. Third, the current design assumes a passive observer and does not offer robustness against active adversaries, which limits direct operational deployment. Any future publication or release should continue to discuss dual-use concerns and avoid overstating the system's practical concealment strength.

C GENERATIVE AI USAGE

This draft was prepared with assistance from a generative AI coding and writing tool. The tool was used to help draft prose, organize sections, and generate initial LaTeX structure based on the implemented repository state. Human authors are responsible for verifying all technical claims, reproducing all experiments, validating all citations, and revising the manuscript before submission.